target value only for the CPU utilization. The Application Auto Scaling service will take care of the rest and will increase or decrease the desired count of the ECS service automatically.

Listing 18.7 Configuring autoscaling based on CPU utilization for the ECS service

```
ScalableTarget:
  Type: AWS::ApplicationAutoScaling::ScalableTarget
  Properties:
    MaxCapacity: '4'                                                    ①
    MinCapacity: '2'                                                    ②
    RoleARN: !GetAtt 'ScalableTargetRole.Arn'                           ③
    ServiceNamespace: ecs                                               ④
    ScalableDimension: 'ecs:service:DesiredCount'                       ⑤
    ResourceId: !Sub                                                    ⑥
    - 'service/${Cluster}/${Service}'
    - Cluster: !Ref Cluster
      Service: !GetAtt 'Service.Name'
CPUScalingPolicy:
  Type: AWS::ApplicationAutoScaling::ScalingPolicy
  Properties:
    PolicyType: TargetTrackingScaling                                   ⑦
    PolicyName: !Sub 'awsinaction-notea-${ApplicationID}'
    ScalingTargetId: !Ref ScalableTarget                                ⑧
    TargetTrackingScalingPolicyConfiguration:
      TargetValue: 50.0                                                 ⑨
      ScaleInCooldown: 180                                              ⑩
      ScaleOutCooldown: 60
      PredefinedMetricSpecification:
        PredefinedMetricType: ECSServiceAverageCPUUtilization
```

① The upper limit for scaling tasks

② The lower limit for scaling tasks

③ The IAM role is required to grant Application Auto Scaling access to CloudWatch metrics and ECS.

④ Application Auto Scaling supports all kinds of services. You want to scale ECS in this example.

⑤ Scales by increasing or decreasing the desired count of the ECS service

⑥ References the ECS service in the ECS cluster created above